

TrustHome PH — System Build Plan (Firebase Edition)

How to actually build every feature in the revised proposal

This explains what each feature does, exactly how its logic works, and how the pieces connect — updated to match your actual stack: **React (JavaScript/TypeScript) for the web app, Firebase (Firestore, Auth, Cloud Functions, Hosting) plus Cloudinary for file storage as the backend**, with React Native as the natural path to a mobile app later since it shares almost all of this same logic.

0. Tech Stack

Layer	Choice	Why
Frontend (web)	React (JavaScript or TypeScript)	Largest ecosystem, fastest with AI-assisted coding, most tutorials to fall back on.
Backend/database	Firebase Firestore	NoSQL document database — no server to manage, generous free tier, built-in real-time updates (handy for live booking status).
Auth	Firebase Authentication	Handles email/password (and optionally Google sign-in) out of the box — don't build this yourself.
File storage	Cloudinary	For listing photos and ID/ownership documents — not Firebase Storage, which now requires billing (see Firebase Setup Plan).
Scheduled jobs	Cloud Functions for Firebase (scheduled functions)	Runs the Trust & Fairness Score recomputation and can trigger the bank scraper on a timer — no separate server needed.
Hosting	Firebase Hosting	Free, simple deploy for the React build — one command from your terminal.
Bank scraper	Python (unchanged)	Scraping is easiest in Python (<code>requests</code> + <code>BeautifulSoup</code>); it writes directly into Firestore using the <code>firebase-admin</code> Python SDK — no conflict with using JS for the web app, these are separate processes.
Future mobile	React Native	Reuses almost all of your Firebase logic (auth calls, Firestore queries, the trust score formula, the booking overlap check) — only the UI layer needs rebuilding.

1. System Overview

TrustHome PH is made of three moving parts:

1. **The React web app** — listings, booking, ratings, disputes, admin review. Talks directly to Firestore/Auth using the Firebase JS SDK, and to Cloudinary for file uploads.
2. **The bank-property scraper** — a standalone Python script (already built) that runs on a schedule and writes into the same Firestore database via `firebase-admin`.
3. **Firestore** — the single shared database both the app and the scraper read/write to.

```
[ Users (browse, list, book) ] ---> [ React Web App ] ---> [ Firestore / Auth ]
                                     ^
[ BDO / Landbank / Metrobank public pages ] --> [ Python Scraper (scheduled) ] --/
```

Keeping the scraper as a separate Python process (not inside the React app) is deliberate: if a bank changes its page layout and breaks the scraper, your core rental-marketplace features keep working uninterrupted.

2. Firestore Data Model

Firestore is a NoSQL **document database** — no tables, no joins, no foreign keys. Instead, you have **collections** (like a folder) full of **documents** (like a JSON object), and relationships are handled by simply storing one document's ID inside another.

Collection	Document fields	Notes
users	name, email, role (<code>user</code> / <code>admin</code>), status (<code>active</code> / <code>suspended</code>), createdAt	Document ID = the Firebase Auth UID — free unique key, no separate ID needed. Renters and homeowners are the same account type (<code>user</code>) — anyone can browse, book, and list a property; only <code>admin</code> is a distinct role.
listings	ownerId, title, description, type, city, price, pricePeriod, floorArea, lotArea, verificationStatus (<code>pending</code> / <code>verified</code> / <code>rejected</code>), verificationDocUrl, photoUrls (array), verifiedAt, createdAt	<code>photoUrls</code> is just an array field — no separate photos collection needed for something this simple.
bookings	listingId, renterId, startDate, endDate, status (<code>Pending</code> / <code>Confirmed</code> / <code>Completed</code> / <code>Disputed</code> / <code>Cancelled</code>), depositReference, createdAt	The whole conflict-check and dispute system hinges on this collection.
ratings	bookingId, raterId, rateId, score (1-5), comment, createdAt	One booking can generate up to two rating documents.
disputes	bookingId, raisedBy, reason, status (<code>Open</code> / <code>Founded</code> / <code>Dismissed</code>), resolutionNotes, resolvedAt, createdAt	Feeds the public accountability flag.
bankProperties	bank, referenceNo, title, location, price, floorArea, lotArea, listingUrl, imageUrl, status (<code>active</code> / <code>delisted</code>), firstSeen, lastSeen	Document ID is set to <code>\${bank}_\${referenceNo}</code> (e.g. <code>bdo_R-2024002140</code>) — this makes the unique-key/dedup logic automatic (see Section 7), instead of needing a manual “check if it exists” query.
trustScores	listingId, score, completedBookingsCount, avgRating, priceFairnessLabel, computedAt	A cache collection, recomputed by a scheduled Cloud Function (see Section 5) so search/sort stays fast.
savedSearches	userId, filters (map: city, minPrice, maxPrice, type), createdAt	Powers the renter-side paid alert subscription.

A quick Firestore mental model if you’re coming from SQL: there’s no `JOIN` — if you need a booking’s listing details, you fetch the booking document, then fetch the listing document using the `listingId` field. Two reads instead of one query. This is normal Firestore practice, not a workaround.

3. Feature: Verified Listing Intake

What it does: Filters out casual fraud before a listing is publicly visible — not a legal title search, and the proposal says so explicitly.

How it works, step by step: 1. A user fills out the listing form to post a property, uploads photos + one ID/ownership document to **Cloudinary**, and creates a `listings` document with `verificationStatus`: `"pending"` — it does not appear in public search yet. (There’s no separate “homeowner” role — any signed-in user can post a listing, the same way any user can browse and book one.) 2. An admin opens a review queue: a React page querying `listings WHERE verificationStatus == "pending"`, showing each document’s uploaded document image (fetched from its Cloudinary URL) alongside the poster’s account name. 3. Admin checks: does the name on the document match the account? Does the document look genuine/complete? A **plausibility check**, not forensic verification — say that plainly if asked. 4. Admin clicks **Approve** → update the document: `verificationStatus`: `"verified"`, `verifiedAt`: `serverTimestamp()`. Or **Reject** → `"rejected"`, with a reason field the owner can see. 5. Public search always queries `WHERE verificationStatus == "verified"`.

Firestore Security Rule to enforce this (not just app logic — enforce it at the database level too):

```
match /listings/{listingId} {
  allow read: if resource.data.verificationStatus == "verified" || request.auth.uid ==
resource.data.ownerId;
  allow write: if request.auth.uid == resource.data.ownerId || isAdmin();
}
```

Implementation notes: This is a manual review workflow by design — no OCR, no ID-verification API, no AI. Fully explainable, zero external dependency, realistic to build within your timeline.

4. Feature: Conflict-Free Booking Calendar

What it does: Makes it impossible to double-book the same listing for overlapping dates.

State machine:

```
Pending → Confirmed → Completed
           ↘ Disputed
           (Cancelled from Pending or Confirmed)
```

The overlap check — Firestore version. Firestore can't run a single query for "does this date range overlap any existing range" the way SQL can (WHERE start < :end AND end > :start isn't expressible as one Firestore query). Instead: fetch the small set of `Confirmed` bookings for that one listing, and check the overlap in your app code:

```
const confirmedSnap = await db.collection('bookings')
  .where('listingId', '=', listingId)
  .where('status', '=', 'Confirmed')
  .get();

const hasConflict = confirmedSnap.docs.some(doc => {
  const b = doc.data();
  return b.startDate < requestedEnd && b.endDate > requestedStart;
});

if (hasConflict) {
  // reject with "these dates are unavailable"
} else {
  // create the new booking document with status: "Pending"
}
```

This is completely normal Firestore practice — a single listing has a small, bounded number of bookings, so fetching them all and checking in code is fast and correct, not a workaround.

Step by step: 1. Renter requests a date range → run the overlap check above against `Confirmed` bookings only (multiple `Pending` requests for the same dates are fine — they're just competing requests, not conflicts yet). 2. Owner approves a `Pending` request → **re-run the same overlap check at confirm-time**, in case two pending requests for the same dates both reached the owner. First confirmed wins; the second gets auto-rejected. 3. On confirmation → status: `"Confirmed"`. 4. After `endDate` passes → a **scheduled Cloud Function** (runs once a day) queries WHERE `endDate < today AND status == "Confirmed"` and transitions them to `"Completed"`. 5. Either party can raise a dispute on a `Confirmed`/`Completed` booking → status: `"Disputed"` (Section 6).

This overlap check is still a textbook interval-intersection problem — the only thing that changed moving to Firestore is *where* the check runs (your app code instead of the database engine), not the logic itself.

5. Feature: Trust & Fairness Score

What it does: Ranks listings by proven trustworthiness instead of just recency or price.

The formula (unchanged):

```
trust_score = (w1 × booking_score) + (w2 × rating_score) + (w3 × fairness_score)
```

1. Booking score — completed bookings, recency-weighted:

```
const completedSnap = await db.collection('bookings')
  .where('listingId', '=', listingId)
  .where('status', '=', 'Completed')
  .get();

const bookingScore = completedSnap.docs.reduce((sum, doc) => {
  const b = doc.data();
  const monthsAgo = (Date.now() - b.endDate.toMillis()) / (1000 * 60 * 60 * 24 * 30);
  return sum + (monthsAgo <= 6 ? 1.0 : 0.5);
}, 0);
```

2. Rating score — average, normalized to 0-1:

```
const ratingsSnap = await db.collection('ratings').where('listingId', '=', listingId).get();
const scores = ratingsSnap.docs.map(d => d.data().score);
const ratingScore = (scores.reduce((a, b) => a + b, 0) / scores.length) / 5;
```

3. Price-fairness score — Firestore has no `MEDIAN()` function, so compute it in code:

```
const comparableSnap = await db.collection('listings')
  .where('city', '=', listing.city)
  .where('type', '=', listing.type)
  .get();
```

```

const pricesPerSqm = comparableSnap.docs
  .map(d => d.data())
  .filter(l => Math.abs(l.floorArea - listing.floorArea) / listing.floorArea <= 0.2)
  .map(l => l.price / l.floorArea)
  .sort((a, b) => a - b);

const median = pricesPerSqm[Math.floor(pricesPerSqm.length / 2)];
const delta = (listing.price / listing.floorArea - median) / median;

let fairnessScore, label;
if (delta > 0.15) { fairnessScore = 0.3; label = "Above market"; }
else if (delta < -0.15) { fairnessScore = 1.0; label = "Below market"; }
else { fairnessScore = 0.8; label = "At market"; }

```

Suggested weights: `w1 = 0.4`, `w2 = 0.4`, `w3 = 0.2` — adjust after testing with sample data.

Anti-gaming safeguard: only `Completed` bookings count (never `Pending` / `Cancelled`); flag for manual review any pattern where the same `renterId` has ≥ 3 completed bookings with the same `ownerId` within 30 days.

Where this actually runs — a scheduled Cloud Function:

```

exports.recomputeTrustScores = functions.pubsub
  .schedule('every 24 hours')
  .onRun(async () => {
    const listingsSnap = await db.collection('listings').where('verificationStatus', '==',
      'verified').get();
    for (const doc of listingsSnap.docs) {
      const score = await computeTrustScore(doc.id, doc.data()); // the three calculations above
      await db.collection('trustScores').doc(doc.id).set(score);
    }
  });

```

Search/sort queries then just read from `trustScores` and `orderBy('score', 'desc')` — fast, and intentional design to mention in your defense.

6. Feature: Deposit-Scam Accountability Trail

What it does: The honest answer to “what stops a verified person from taking a deposit and disappearing?”

Step by step: 1. Renter and owner arrange the deposit directly (GCash, bank transfer). TrustHome never touches this money. 2. On the `Confirmed` booking document, the renter fills in `depositReference` — a simple field update. 3. If the owner doesn't honor the booking, the renter creates a `disputes` document: `{ bookingId, raisedBy, reason, status: "Open" }`. 4. Admin reviews it (same review-queue pattern as Section 3) → updates `status` to `"Founded"` or `"Dismissed"`. 5. If `"Founded"`: - Update the owner's `users` document: `status: "suspended"`. - The public flag is simply **derived at display time** — anywhere you show that user's profile, run a quick count of `disputes` with `status == "Founded"` scoped to bookings on that owner's listings, and show “ Δ N founded dispute(s)” if count > 0 . No separate “flag” field needed — it's computed from the `disputes` collection directly, so it can never go stale or be forgotten. 6. This is what you show the panel when asked “what actually stops the scam.”

Say this explicitly in your defense: “We do not custody funds. We provide a verifiable accountability trail and a public consequence system — that's the scope we can fully build and defend within our timeline.”

7. Feature: Bank-Acquired Properties Catalog

Already built — the Python scraper writes into the `bankProperties` collection. **Update needed:** swap the storage layer from SQLite to Firestore using `firebase-admin`. The unique-key logic actually gets *simpler* in Firestore, because you can set the document ID directly instead of querying to check existence first:

```

import firebase_admin
from firebase_admin import credentials, firestore

cred = credentials.Certificate("serviceAccountKey.json")
firebase_admin.initialize_app(cred)
db = firestore.client()

def upsert_properties(bank: str, scraped_listings: list[dict]):
    now = firestore.SERVER_TIMESTAMP
    scraped_refs = set()

    for item in scraped_listings:
        doc_id = f"{bank}_{item['reference_no']}" # <-- this IS the unique key
        scraped_refs.add(doc_id)
        doc_ref = db.collection('bankProperties').document(doc_id)
        doc_ref.set({
            **item,
            "bank": bank,
            "status": "active",
            "lastSeen": now,

```

```

    }, merge=True) # merge=True: update if exists, create if not – no separate insert/update
    branch needed

    # Flag anything that used to be active but wasn't in this scrape run
    existing = db.collection('bankProperties').where('bank', '==', bank).where('status', '==',
'active').stream()
    for doc in existing:
        if doc.id not in scraped_refs:
            doc.reference.update({"status": "delisted", "lastSeen": now})

```

This is actually cleaner than the SQL version — `set(..., merge=True)` with a deterministic document ID collapses the “check if exists, then insert or update” logic into one call.

What the React app needs to do with it: 1. A read-only, filterable page — query `bankProperties` filtered by `bank`, `city`, price range. 2. Each entry displays `lastSeen` as “**Last verified: [date]**”. 3. Links to the loan calculator (Section 8), pre-filled with that property’s price.

Scheduling the scraper: since it’s a Python script (not JS), it can’t run *inside* a Firebase Cloud Function directly. Simplest options: run it on a schedule via **GitHub Actions** (as already set up in your `bank_scraper` project — just point its output at Firestore now instead of SQLite), or trigger it from a scheduled Cloud Function that calls out to wherever it’s hosted. GitHub Actions is simpler and still completely free — no need to overcomplicate this.

8. Feature: Loan / Amortization Calculator

What it does: Self-contained, no external dependency — unchanged by the move to Firebase, since it’s pure computation with no database involved.

The formula:

```

P = property_price - down_payment
r = annual_interest_rate / 12
n = loan_term_years * 12

monthly_payment = P * [ r * (1 + r)^n ] / [ (1 + r)^n - 1 ]

```

React implementation: a plain JavaScript function, called directly from the component — no Firestore read/write needed at all:

```

function calculateMonthlyPayment(price, downPaymentPercent, annualRate, termYears) {
    const P = price - (price * downPaymentPercent);
    const r = annualRate / 12;
    const n = termYears * 12;
    return P * (r * Math.pow(1 + r, n)) / (Math.pow(1 + r, n) - 1);
}

```

How to validate it works: plug the same inputs into any existing online mortgage calculator and confirm your output matches.

9. Suggested Build Order

Phase	Build	Why this order
1. Foundations	Firebase project setup (Firestore, Auth, Storage), React app scaffold, listing CRUD + photo/document upload, basic search	Nothing else works without this.
2. Booking Calendar	Booking request flow, overlap-check logic, status transitions	Your first real computational feature.
3. Trust & Fairness Score	Ratings, price-fairness calculation, scheduled Cloud Function, ranked search	Needs seed/sample data to look convincing in a demo.
4. Dispute Trail	Dispute submission, admin review queue, derived public flag	Builds directly on Phase 2.
5. Bank Catalog + Loan Calculator	Point the existing scraper at Firestore, build the catalog page, add the calculator	Fully independent — can be built in parallel.
6. Polish & Testing	Seed realistic sample data, fix edge cases, prepare demo script	Where <code>Test_Plan.xlsx</code> and <code>Test_Case_Template.docx</code> get filled in.

10. Demo-Day Sample Data Checklist

- Seed 15–20 listings across a few cities in Firestore, mixed `verificationStatus` (mostly `verified`, a couple `pending` to show the review queue works).
- Seed enough `Completed` bookings and ratings so the Trust & Fairness ranking visibly differs from a plain recency/price sort.
- Seed one deliberately overlapping booking attempt to demo the conflict rejection live.
- Seed one `Founded` dispute so the public flag is visible on a profile.
- Run the scraper once before demo day so `bankProperties` has real, current entries with a real `Last verified` date.
- A Firestore-specific tip: use the **Firestore Emulator Suite** during development so you're not burning through your free-tier quota or risking messing up real seeded demo data while testing — switch to the real project only for the actual demo.